

Login System with Validation

Full Stack Development – TASK_3

Authentication Module with Client-side Validation

1 Introduction

The Login System with Validation is a web-based authentication module developed using HTML5, CSS3, Bootstrap, JavaScript, Node.js, Express.js, and MySQL. It validates user inputs on the client side, checks credentials from the database, and displays dynamic error messages without page reload.

2 Software Requirements

- Node.js
- MySQL Server
- VS Code
- Web Browser
- Git (Optional)

3 Project Structure

```
login-system/  
  
  server.js  
  package.json  
  
  public/  
    css/  
      style.css  
    js/  
      login.js  
      login.html  
  
  views/  
    dashboard.ejs
```

4 Database Design

Database Name: login_db

Table Name: users

```
1 CREATE DATABASE login_db;  
2 USE login_db;  
3  
4 CREATE TABLE users (
```

```

5     id INT AUTO_INCREMENT PRIMARY KEY,
6     email VARCHAR(100) NOT NULL,
7     password VARCHAR(100) NOT NULL
8 );
9
10 INSERT INTO users (email, password)
11 VALUES ('admin@gmail.com', '123456');

```

Listing 1: Database and Users Table Creation

5 server.js Code

```

1 const express = require('express');
2 const mysql = require('mysql2');
3 const bodyParser = require('body-parser');
4 const path = require('path');
5
6 const app = express();
7 app.use(bodyParser.json());
8 app.use(express.static('public'));
9 app.set('view engine', 'ejs');
10
11 const db = mysql.createConnection({
12     host: 'localhost',
13     user: 'root',
14     password: 'yourpassword',
15     database: 'login_db'
16 });
17
18 db.connect((err) => {
19     if (err) throw err;
20     console.log('MySQL Connected...');
21 });
22
23 // Serve login page
24 app.get('/', (req, res) => {
25     res.sendFile(path.join(__dirname, 'public', 'login.html'));
26 });
27
28 // Handle login request
29 app.post('/login', (req, res) => {
30     const { email, password } = req.body;
31
32     const sql = 'SELECT * FROM users WHERE email = ? AND password = ?';
33     db.query(sql, [email, password], (err, results) => {
34         if (err) {
35             return res.status(500).json({ error: 'Database error' });
36         }
37
38         if (results.length > 0) {
39             res.json({ success: true });
40         } else {
41             res.json({ success: false, message: 'Invalid email or password' });
42         }
43     });
44 });
45
46 // Dashboard route
47 app.get('/dashboard', (req, res) => {
48     res.render('dashboard');
49 });
50
51 app.listen(3000, () => {
52     console.log('Server running on port 3000');

```

6 login.html Code

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Login System</title>
7   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel
   ="stylesheet">
8   <link rel="stylesheet" href="/css/style.css">
9 </head>
10 <body class="bg-light">
11   <div class="container">
12     <div class="row justify-content-center mt-5">
13       <div class="col-md-4">
14         <div class="card shadow">
15           <div class="card-header bg-primary text-white text-center">
16             <h4>Login System</h4>
17           </div>
18           <div class="card-body">
19             <div id="errorMessage" class="alert alert-danger d-none"></div>
20
21             <form id="loginForm">
22               <div class="mb-3">
23                 <label for="email" class="form-label">Email address</label>
24                 <input type="email" class="form-control" id="email" required
25
26               <div class="invalid-feedback">Please enter a valid email</
27               </div>
28
29               <div class="mb-3">
30                 <label for="password" class="form-label">Password</label>
31                 <input type="password" class="form-control" id="password"
32                 required>
33
34                 <div class="invalid-feedback">Password is required</div>
35                 </div>
36
37                 <button type="submit" class="btn btn-primary w-100">Login</
38                 button>
39               </form>
40
41               <div class="mt-3 text-center">
42                 <small>Demo credentials: admin@gmail.com / 123456</small>
43               </div>
44             </div>
45           </div>
46         </div>
47       </div>
48     </div>
49   </div>
50   <script src="/js/login.js"></script>
51 </body>
52 </html>

```

Listing 3: Login Form with Bootstrap

7 login.js Code

```

1 document.getElementById('loginForm').addEventListener('submit', async function(e) {
2     e.preventDefault();
3
4     // Get form values
5     const email = document.getElementById('email').value.trim();
6     const password = document.getElementById('password').value;
7
8     // Reset previous errors
9     document.getElementById('errorMessage').classList.add('d-none');
10    document.getElementById('email').classList.remove('is-invalid');
11    document.getElementById('password').classList.remove('is-invalid');
12
13    // Client-side validation
14    let isValid = true;
15
16    if (!email || !isValidEmail(email)) {
17        document.getElementById('email').classList.add('is-invalid');
18        isValid = false;
19    }
20
21    if (!password) {
22        document.getElementById('password').classList.add('is-invalid');
23        isValid = false;
24    }
25
26    if (!isValid) return;
27
28    try {
29        // Send login request
30        const response = await fetch('/login', {
31            method: 'POST',
32            headers: {
33                'Content-Type': 'application/json'
34            },
35            body: JSON.stringify({ email, password })
36        });
37
38        const data = await response.json();
39
40        if (data.success) {
41            window.location.href = '/dashboard';
42        } else {
43            showError(data.message || 'Invalid credentials');
44        }
45    } catch (error) {
46        showError('Connection error. Please try again.');

```

Listing 4: Client-side Validation and AJAX Login

8 dashboard.ejs Code

```

1 <!DOCTYPE html>
2 <html lang="en">

```

```

3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Dashboard</title>
7   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel
   ="stylesheet">
8 </head>
9 <body>
10   <nav class="navbar navbar-dark bg-primary">
11     <div class="container">
12       <span class="navbar-brand mb-0 h1">Login System</span>
13     </div>
14   </nav>
15
16   <div class="container mt-5">
17     <div class="row">
18       <div class="col-md-8 mx-auto">
19         <div class="card shadow">
20           <div class="card-body text-center">
21             <h2 class="text-success">Welcome to Dashboard!</h2>
22             <p class="lead">You have successfully logged in.</p>
23             <hr>
24             <p>This is a protected page that can only be accessed after
authentication.</p>
25             <a href="/" class="btn btn-primary">Logout</a>
26           </div>
27         </div>
28       </div>
29     </div>
30   </div>
31 </body>
32 </html>

```

Listing 5: Welcome Dashboard After Successful Login

9 style.css Code

```

1 body {
2   background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
3   min-height: 100vh;
4 }
5
6 .card {
7   border: none;
8   border-radius: 10px;
9 }
10
11 .card-header {
12   border-radius: 10px 10px 0 0 !important;
13 }
14
15 .btn-primary {
16   background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
17   border: none;
18   transition: transform 0.3s;
19 }
20
21 .btn-primary:hover {
22   transform: translateY(-2px);
23   background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
24 }
25
26 .form-control:focus {
27   border-color: #764ba2;
28   box-shadow: 0 0 0 0.2rem rgba(118, 75, 162, 0.25);
29 }

```

```

30
31 .invalid-feedback {
32     display: block;
33 }
34
35 .alert {
36     margin-bottom: 20px;
37 }

```

Listing 6: Custom CSS Styling

10 Features Implemented

- **Client-side Validation** using JavaScript
 - Email format validation using regex
 - Password field validation (non-empty)
 - Real-time visual feedback with Bootstrap classes
- **Dynamic Error Messages**
 - No page reload - AJAX based communication
 - User-friendly error display
 - Form field highlighting for invalid inputs
- **Bootstrap Responsive UI**
 - Mobile-friendly design
 - Gradient background styling
 - Card-based layout with shadow effects
- **Database Authentication**
 - MySQL database connectivity
 - Secure credential verification
 - Pre-inserted demo user
- **Session Management**
 - Redirect to dashboard after successful login
 - Protected routes concept
 - Logout functionality

11 Validation Rules

Table 1: Client-side Validation Rules

Field	Validation Rule	Error Message
Email	Required + Valid Format	"Please enter a valid email"
Password	Required (non-empty)	"Password is required"
Credentials	Match database record	"Invalid email or password"

12 Steps to Execute

1. **Initialize Node project:**

```
npm init -y
```

2. Install required packages:

```
npm install express mysql2 body-parser ejs
```

3. Set up MySQL database:

- Open MySQL Workbench
- Run the SQL script provided in Section 4
- Verify the users table with demo data

4. Update database credentials in server.js:

- Change 'yourpassword' to your MySQL password

5. Run the server:

```
node server.js
```

6. Access the application:

```
http://localhost:3000
```

7. Test the login:

- Use email: admin@gmail.com
- Use password: 123456
- Try invalid credentials to see error messages

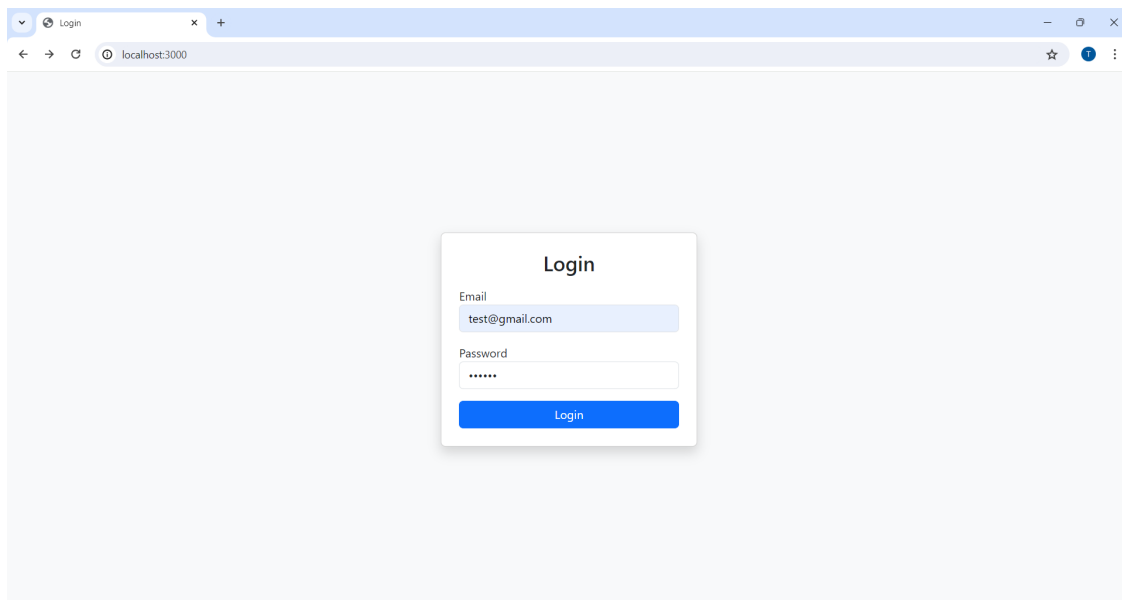


Figure 1: login page

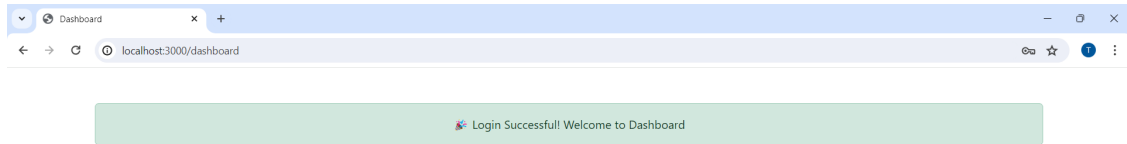


Figure 2: Dashboard

13 Testing Scenarios

Scenario 1: Empty Fields

- Leave both fields empty and submit
- Expected: Both fields show red border with error messages

Scenario 2: Invalid Email Format

- Enter: "admin" (without @domain)
- Expected: Email field shows validation error

Scenario 3: Wrong Credentials

- Email: admin@gmail.com
- Password: wrongpassword
- Expected: Error message "Invalid email or password" appears

Scenario 4: Successful Login

- Email: admin@gmail.com
- Password: 123456
- Expected: Redirect to dashboard page

14 Conclusion

The Login System successfully demonstrates full stack authentication including frontend validation, backend processing, database connectivity, and dynamic error handling. This system forms the foundation of secure authentication in real-time web applications.

Key Learning Outcomes

- Client-side validation techniques with JavaScript
- AJAX communication without page reload
- Database integration for user authentication
- Bootstrap styling for modern UI
- Error handling and user feedback mechanisms